How We Built This Project

Nanoparticle Toxicity Prediction
explained in simple words

A short, plain-English walkthrough

Team Member	Enrolment No.
Pratham Joshi	KU2507U0361
Divyansh Khatri	KU2507U0188
Dhyey Bhatt	KU2507U0138
Jinay Shah	KU2507U0070

Start Here: What Is This Project?

In one sentence: **we built a computer program that looks at a tiny particle and guesses whether it is poisonous to human cells.**

What is a nanoparticle?

It is an extremely small piece of material. To get a sense of the scale: if a nanoparticle were the size of a football, then a football would be the size of the Earth. They are measured in **nanometres** — a nanometre is one millionth of a millimetre.

Why do doctors care about them?

Because they are so small, they can be used to **carry medicine directly to a sick cell** — for example, delivering a cancer drug straight to a tumour instead of flooding the whole body with it. Fewer side effects, better results.

So what is the problem?

The same smallness that makes them useful also makes them dangerous. They can slip into healthy cells too, and some of them poison those cells. So before using any new particle in medicine, scientists must check: **is this one safe or is it toxic?**

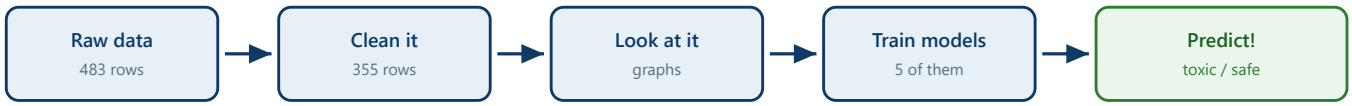
Why not just test them in a lab?

They do — but lab testing is slow and expensive. There are thousands of possible particle designs (different materials, sizes, coatings, doses). Testing every single one is impossible.

> **Our idea:** use a computer to **quickly guess** which particles are likely toxic, so scientists only spend lab time and money on the ones worth testing. Think of it as a filter, not a replacement for the lab.

The Big Picture

The whole project is really just five stages:



Everything below is just these five boxes explained in more detail.

What is "machine learning", in simple words?

Normally you program a computer with rules: "if X, then do Y." Machine learning is different. You do not give it rules — you give it **lots of examples**, and it figures out the rules by itself.

> **Everyday example:** nobody wrote a rule that says "an email containing the word FREE five times is spam." Your email app learned that by looking at millions of emails that people marked as spam. Our project works exactly the same way — we showed the computer hundreds of nanoparticle experiments and let it work out the pattern.

Step By Step: How We Actually Did It

1 We found the data

A machine learning model is useless without examples to learn from. We found a **free, published dataset** from a real scientific study. It contains **483 laboratory experiments** on nanoparticles.

Each experiment recorded things like:

- What material the particle was made of (5 different materials)
- How big it was
- Its surface charge and surface area
- How much of it was given to the cells (the **dose**)
- How long the cells were exposed to it (the **exposure time**)
- And finally: **did the cells die or not?**

That last one is the answer we want the computer to learn to predict.

2 We looked at the data before touching it

Before changing anything, we just printed out what was in the file. How many rows? Any blanks? Any repeated rows? How many toxic vs safe?

This is where we found something important: **only about 1 in 5 experiments was toxic.** The rest were safe. That sounds like a small detail, but it causes a big trap later (see Step 6).

3 We cleaned the data — and found two traps

Real data is always messy. Cleaning it is boring but it is the most important step in the whole project. We found two serious problems.

> **Trap 1 — the answer was hidden in the question.**
> The file had a column called *viability*, which means "what percentage of cells survived." But the toxic/safe label was **calculated from that very column** by the original scientists.
>
> So if we let the computer see it, the computer is not predicting anything — it is just reading the answer.
>
> **Like this:** imagine giving a student an exam, but accidentally printing the answer key on the back

of the paper. They score 100%, and you have learned nothing about whether they can actually do the work.

What we did: deleted that column completely.

Trap 2 — the same experiment appeared many times.
Scientists repeat experiments to be careful. So the same test (same particle, same dose, same cells) appears several times in the file, each with a slightly different survival number.

Once we deleted the *viability* column, those repeated rows became **exactly identical**. We found **122 such repeated rows**.

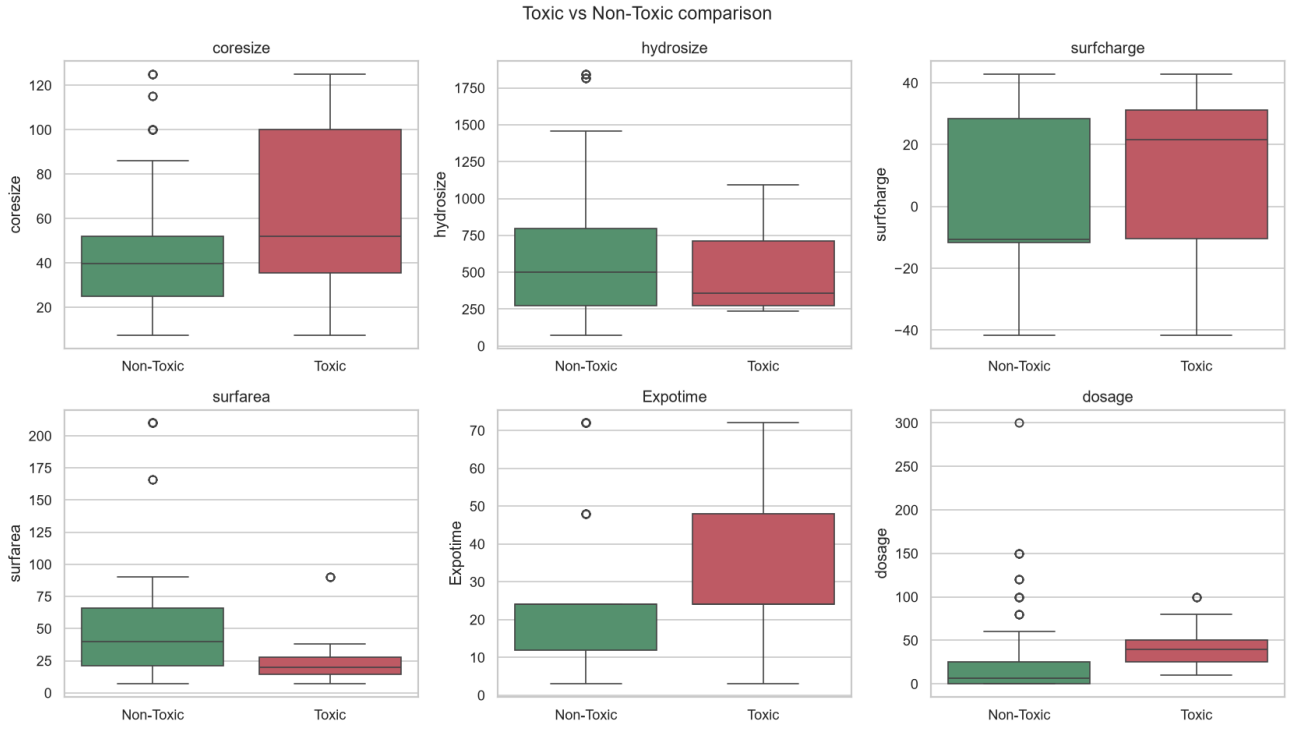
Why this is bad: later we split the data into "practice questions" and "exam questions." If the same experiment appears in both, the computer has already seen the exam answers during practice. Its score looks great but it is cheating without knowing it.

What we did: merged the repeats so each unique experiment appears only once. When repeats disagreed with each other, we took the majority answer.

Result of cleaning: 483 rows became **355 genuine, unique experiments**. Smaller, but honest.

4 We drew graphs to understand it

Before building anything, we plotted the data to see patterns with our own eyes. This is called **EDA** (Exploratory Data Analysis) — a fancy name for "look at your data properly."



Green = safe particles, red = toxic ones, compared property by property.

What we could see straight away:

- **Higher dose → more toxic.** Not surprising, but good — it means the data behaves sensibly.
- **Longer exposure → more toxic.** Same idea.
- Two materials in particular (**CuO** and **ZnO**) caused almost all the toxicity. The other three were basically harmless.

5 We split the data into practice and exam

We divided the 355 experiments into two piles:

- **266 experiments for learning** (75%) — the practice questions
- **89 experiments kept hidden** (25%) — the exam

> **Why hide some?** If we tested the computer on the same examples it learned from, of course it would do well — it memorised them. The only fair test is on experiments it has **never seen before**. That is exactly how a real exam works.

6 We trained five different models

There is no single "best" machine learning method, so we tried five common ones and let the results decide:

- **Logistic Regression** — draws a straight dividing line between toxic and safe
- **Decision Tree** — a flowchart of yes/no questions
- **K-Nearest Neighbours** — "which past experiments look most like this one?"
- **Support Vector Machine** — draws a curved dividing boundary
- **Random Forest** — hundreds of decision trees that vote together

> **The trap we avoided here.** Remember that only 1 in 5 experiments is toxic? That means a completely useless model that just says **"safe"** to absolutely everything would still be right **81% of the time**.
>
> **Like this:** a doctor who tells every single patient "you're fine, go home" will be correct most of the time, because most people are healthy. That does not make them a good doctor. The whole point is catching the sick ones.
>
> **What we did:** we told the models to pay extra attention to the rare toxic cases, and we stopped judging them by accuracy alone.

7 We tested them properly

We gave all five models the 89 hidden experiments and measured them in several ways, not just one:

Measure	What it means in plain words
Accuracy	Out of everything, how often was it right?
Recall	Out of the truly toxic particles, how many did it catch? *(most important for us)*
Precision	When it shouted "toxic!", how often was it actually right?
F1 score	A fair combination of recall and precision

> **Why recall matters most here:** there are two ways to be wrong. Calling a safe particle toxic just wastes one lab test. Calling a **toxic particle safe** could let something harmful move forward. The second mistake is far worse, so we care most about catching every toxic one.

8 We picked the winner

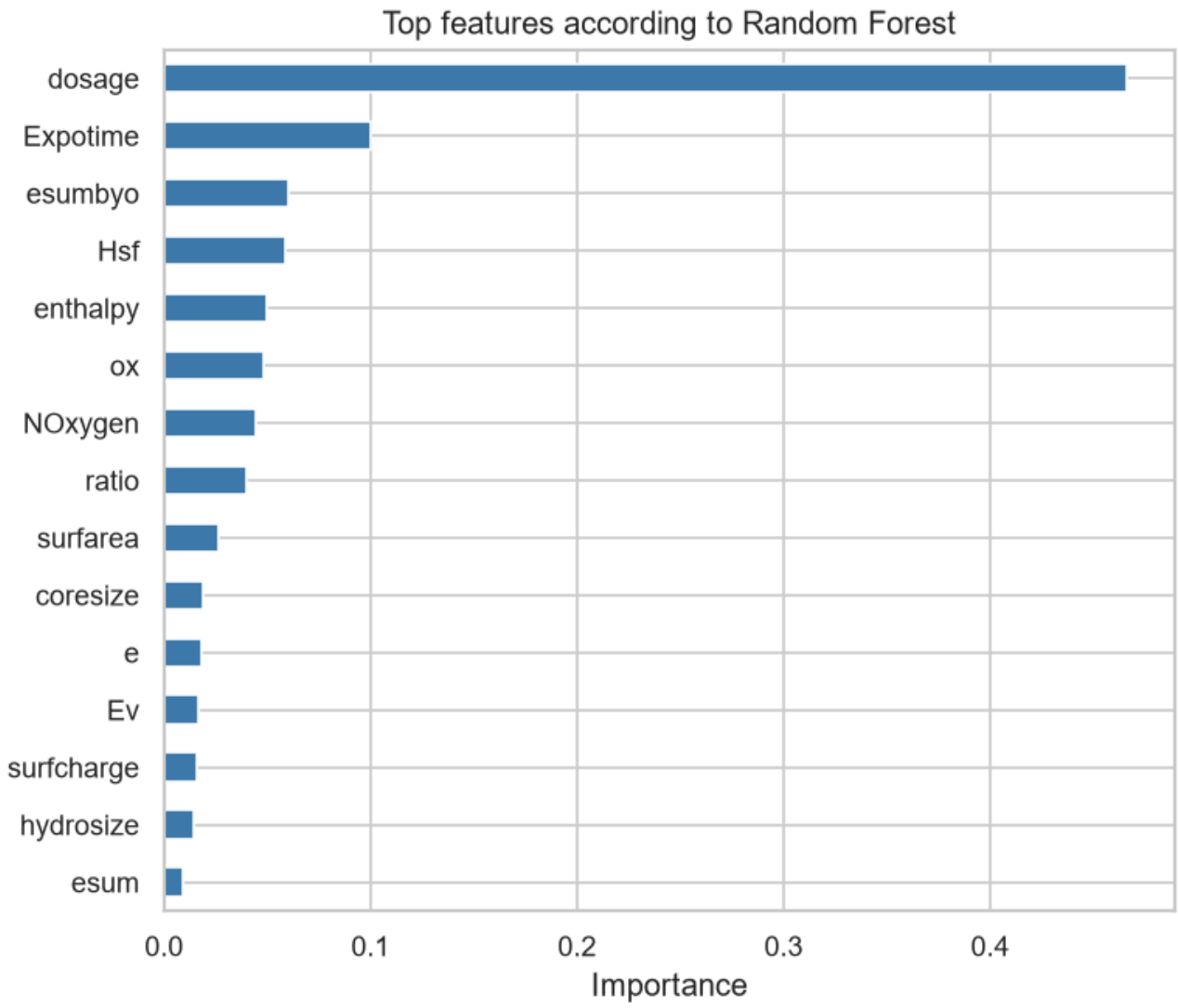
Model	Accuracy	Recall	F1
Random Forest	**94.4%**	**94.1%**	**0.865**
Decision Tree	92.1%	82.4%	0.800
Logistic Regression	88.8%	82.4%	0.737
Support Vector Machine	86.5%	82.4%	0.700
K-Nearest Neighbours	87.6%	41.2%	0.560

Random Forest won. It gets 94.4% right overall, and more importantly it catches **94% of the toxic particles**.

> **Look at the last row.** K-Nearest Neighbours has 87.6% accuracy — that looks respectable! But its recall is only 41.2%, meaning it **misses nearly 6 out of every 10 toxic particles**.
>
> This is exactly the "lazy doctor" problem. If we had picked our model by accuracy alone, we might have chosen this one and never realised it barely works.

9 We asked: which properties actually matter?

A good model should also teach us something. So we asked the Random Forest which pieces of information it relied on most.



The longer the bar, the more the model depends on that property.

The answer was clear:

- **Dose matters most by far** — how much you give is the single biggest factor.
- **Exposure time** comes second.
- Then the **chemistry of the material** itself.
- Particle size mattered surprisingly little.

> **In plain terms: what the particle is made of sets the danger level, and how much you use decides whether that danger actually shows up.**

10 We built something people can actually use

A model sitting in a file is not useful. So we built two ways to use it:

- A **web app** — you pick the material, type in six numbers, press a button, and it tells you TOXIC or NON-TOXIC with a confidence percentage.
- A **command line tool** — the same thing for people comfortable with a terminal.

The user only has to enter **six numbers**. The other thirteen chemistry values are filled in automatically once you pick the material, because those never change for a given substance.

11 We tried to break our own project

Finally, instead of assuming it worked, we deliberately went hunting for mistakes. We wrote **18 automatic tests** and found **7 problems**.

The most interesting one: a test we *expected to pass* failed, and that is how we discovered the repeated-experiments problem from Step 3. Fixing it actually **changed which model won** — the Decision Tree had been coming first only because it was quietly memorising the duplicate rows.

> **Lesson:** testing your own work is not extra credit. It changed our actual result.

What We Found

Question	Answer
How much data did we end up with?	355 clean experiments (from 483 raw)
How many were toxic?	68 out of 355, about 1 in 5
Which model is best?	Random Forest
How accurate is it?	94.4%
How many toxic particles does it catch?	94 out of every 100
What matters most for toxicity?	The dose, then exposure time, then the material
Which materials were dangerous?	CuO and ZnO. Al_2O_3, Fe_2O_3 and TiO_2 were nearly harmless

What Our Project Cannot Do

Being honest about limits is part of doing this properly.

> **It only knows five materials.** Our data covered just five substances. We tested what happens with a completely new material by hiding one material entirely and asking the model to judge it blind. Its accuracy dropped from **94% to 77%**.
>
> So the model is reliable for materials similar to the ones it studied, but you **should not trust it on a brand-new substance it has never seen**. Fixing this needs a bigger dataset, not better code.

- 355 experiments is a **small** dataset by machine learning standards.
- The tests were done on cells in a dish, which is not the same as a whole human body.
- It is a **screening tool** — a helpful filter to decide what to test in the lab, not a replacement for the lab.

The Whole Project In One Paragraph

We took 483 real laboratory experiments on nanoparticles and cleaned them down to 355 trustworthy ones, throwing away a column that was secretly giving away the answer and merging repeated experiments that would have let the computer cheat. We looked at the data with graphs, split it into practice and exam portions, and trained five different machine learning models on it. The Random Forest performed best, correctly identifying 94% of toxic particles on experiments it had never seen. It told us that dose and exposure time matter most, followed by the chemistry of the material. We wrapped it in an app anyone can use, then wrote 18 tests to hunt for our own mistakes — and found seven, one of which changed our final answer.

Quick Word List

Term	Simple meaning
Nanoparticle	An incredibly tiny piece of material
Toxic	Harmful — it kills cells
Dataset	The table of past experiments we learn from
Feature	One piece of information about a particle (its size, its dose...)
Label	The answer we are trying to predict (toxic or safe)
Training	Letting the computer study the examples
Model	The thing that comes out of training and makes predictions
Data leakage	Accidentally showing the answer during training — cheating by mistake
Accuracy	How often it is right overall
Recall	How many of the truly toxic ones it managed to catch
EDA	Drawing graphs to understand your data before modelling